Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §5.2.7.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.7.

3.7 Interpolation on Scattered Data in Multidimensions

We now leave behind, if with some trepidation, the orderly world of regular grids. Courage is required. We are given an arbitrarily scattered set of N data points $(\mathbf{x}_i, y_i), i = 0, \ldots, N-1$ in n-dimensional space. Here $\mathbf{x}_i$ denotes an n-dimensional vector of independent variables, $(x_{1i}, x_{2i}, \ldots, x_{ni})$, and y_i is the value of the function at that point.

In this section we discuss two of the most widely used *general* methods for this problem, *radial basis function (RBF)* interpolation, and *kriging*. Both of these methods are expensive. By that we mean that they require $O(N^3)$ operations to initially digest a set of data points, followed by $O(N)$ operations for each interpolated value. Kriging is also able to supply an error estimate — but at the rather high cost of $O(N^2)$ per value. Shepard interpolation, discussed below, is a variant of RBF that at least avoids the $O(N^3)$ initial work; otherwise these workloads effectively limit the usefulness of these general methods to values of $N \lesssim 10^4$. It is therefore worthwhile for you to consider whether you have any other options. Two of these are

- If n is not too large (meaning, usually, $n = 2$), and if the data points are fairly dense, then consider triangulation, discussed in §21.6. Triangulation is an example of a finite element method. Such methods construct some semblance of geometric regularity and then exploit that construction to advantage. Mesh generation is a closely related subject.

- If your accuracy goals will tolerate it, consider moving each data point to the nearest point on a regular Cartesian grid and then using Laplace interpolation (§3.8) to fill in the rest of the grid points. After that, you can interpolate on the grid by the methods of §3.6. You will need to compromise between making the grid very fine (to minimize the error introduced when you move the points) and the compute time workload of the Laplace method.

If neither of these options seem attractive, and you can't think of another one that is, then try one or both of the two methods that we now discuss. RBF interpolation is probably the more widely used of the two, but kriging is our personal favorite. Which works better will depend on the details of your problem.

The related, but easier, problem of *curve interpolation* in multidimensions is discussed at the end of this section.

3.7.1 Radial Basis Function Interpolation

The idea behind RBF interpolation is very simple: Imagine that every known point j "influences" its surroundings the same way in all directions, according to

some assumed functional form $\phi(r)$ — the radial basis function — that is a function only of radial distance $r = |\mathbf{x} - \mathbf{x}_j|$ from the point. Let us try to approximate the interpolating function everywhere by a linear combination of the ϕ's, centered on all the known points,

$$y(\mathbf{x}) = \sum_{i=0}^{N-1} w_i \phi(|\mathbf{x} - \mathbf{x}_i|) \tag{3.7.1}$$

where the w_i's are some unknown set of weights. How do we find these weights? Well, we haven't used the function values y_i yet. The weights are determined by requiring that the interpolation be exact at all the known data points. That is equivalent to solving a set of N linear equations in N unknowns for the w_i's:

$$y_j = \sum_{i=0}^{N-1} w_i \phi(|\mathbf{x}_j - \mathbf{x}_i|) \tag{3.7.2}$$

For many functional forms ϕ, it can be proved, under various general assumptions, that this set of equations is nondegenerate and can be readily solved by, e.g., LU decomposition (§2.3). References [1,2] provide entry to the literature.

A variant on RBF interpolation is *normalized radial basis function (NRBF)* interpolation, in which we require the sum of the basis functions to be unity or, equivalently, replace equations (3.7.1) and (3.7.2) by

$$y(\mathbf{x}) = \frac{\sum_{i=0}^{N-1} w_i \phi(|\mathbf{x} - \mathbf{x}_i|)}{\sum_{i=0}^{N-1} \phi(|\mathbf{x} - \mathbf{x}_i|)} \tag{3.7.3}$$

and

$$y_j \sum_{i=0}^{N-1} \phi(|\mathbf{x}_j - \mathbf{x}_i|) = \sum_{i=0}^{N-1} w_i \phi(|\mathbf{x}_j - \mathbf{x}_i|) \tag{3.7.4}$$

Equations (3.7.3) and (3.7.4) arise more naturally from a Bayesian statistical perspective [3]. However, there is no evidence that either the NRBF method is consistently superior to the RBF method, or vice versa. It is easy to implement both methods in the same code, leaving the choice to the user.

As we already mentioned, for N data points the one-time work to solve for the weights by LU decomposition is $O(N^3)$. After that, the cost is $O(N)$ for each interpolation. Thus $N \sim 10^3$ is a rough dividing line (at 2007 desktop speeds) between "easy" and "difficult." If your N is larger, however, don't despair: There are *fast multipole methods*, beyond our scope here, with much more favorable scaling [1,4,5]. Another, much lower-tech, option is to use *Shepard interpolation* discussed later in this section.

Here are a couple of objects that implement everything discussed thus far. `RBF_fn` is a virtual base class whose derived classes will embody different functional forms for `rbf`$(r) \equiv \phi(r)$. `RBF_interp`, via its constructor, digests your data and solves the equations for the weights. The data points $\mathbf{x}_i$ are input as an $N \times n$ matrix, and the code works for any dimension n. A boolean argument `nrbf` inputs whether NRBF is to be used instead of RBF. You call `interp` to get an interpolated function value at a new point $\mathbf{x}$.

interp_rbf.h

```
struct RBF_fn {
```
Abstract base class template for any particular radial basis function. See specific examples below.
```
    virtual Doub rbf(Doub r) = 0;
};

struct RBF_interp {
```
Object for radial basis function interpolation using `n` points in `dim` dimensions. Call constructor once, then `interp` as many times as desired.
```
    Int dim, n;
    const MatDoub &pts;
    const VecDoub &vals;
    VecDoub w;
    RBF_fn &fn;
    Bool norm;

    RBF_interp(MatDoub_I &ptss, VecDoub_I &valss, RBF_fn &func, Bool nrbf=false)
    : dim(ptss.ncols()), n(ptss.nrows()) , pts(ptss), vals(valss),
    w(n), fn(func), norm(nrbf) {
```
Constructor. The $n \times$ `dim` matrix `ptss` inputs the data points, the vector `valss` the function values. `func` contains the chosen radial basis function, derived from the class `RBF_fn`. The default value of `nrbf` gives RBF interpolation; set it to 1 for NRBF.
```
        Int i,j;
        Doub sum;
        MatDoub rbf(n,n);
        VecDoub rhs(n);
        for (i=0;i<n;i++) {
```
Fill the matrix $\phi(|\mathbf{r}_i - \mathbf{r}_j|)$ and the r.h.s. vector.
```
            sum = 0.;
            for (j=0;j<n;j++) {
                sum += (rbf[i][j] = fn.rbf(rad(&pts[i][0],&pts[j][0])));
            }
            if (norm) rhs[i] = sum*vals[i];
            else rhs[i] = vals[i];
        }
        LUdcmp lu(rbf);
```
Solve the set of linear equations.
```
        lu.solve(rhs,w);
    }

    Doub interp(VecDoub_I &pt) {
```
Return the interpolated function value at a `dim`-dimensional point `pt`.
```
        Doub fval, sum=0., sumw=0.;
        if (pt.size() != dim) throw("RBF_interp bad pt size");
        for (Int i=0;i<n;i++) {
```
Sum over all tabulated points.
```
            fval = fn.rbf(rad(&pt[0],&pts[i][0]));
            sumw += w[i]*fval;
            sum += fval;
        }
        return norm ? sumw/sum : sumw;
    }

    Doub rad(const Doub *p1, const Doub *p2) {
```
Euclidean distance.
```
        Doub sum = 0.;
        for (Int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
        return sqrt(sum);
    }
};
```

3.7.2 Radial Basis Functions in General Use

The most often used radial basis function is the *multiquadric* first used by Hardy, circa 1970. The functional form is

$$\phi(r) = (r^2 + r_0^2)^{1/2} \tag{3.7.5}$$

where r_0 is a scale factor that you get to choose. Multiquadrics are said to be less sensitive to the choice of r_0 than some other functional forms.

In general, both for multiquadrics and for other functions, below, r_0 should be larger than the typical separation of points but smaller than the "outer scale" or feature size of the function that you are interpolating. There can be several orders of magnitude difference between the interpolation accuracy with a good choice for r_0, versus a poor choice, so it is definitely worth some experimentation. One way to experiment is to construct an RBF interpolator omitting one data point at a time and measuring the interpolation error at the omitted point.

The *inverse multiquadric*

$$\phi(r) = (r^2 + r_0^2)^{-1/2} \tag{3.7.6}$$

gives results that are comparable to the multiquadric, sometimes better.

It might seem odd that a function and its inverse (actually, reciprocal) work about equally well. The explanation is that what really matters is smoothness, and certain properties of the function's Fourier transform that are not very different between the multiquadric and its reciprocal. The fact that one increases monotonically and the other decreases turns out to be almost irrelevant. However, if you want the extrapolated function to go to zero far from all the data (where an accurate value is impossible anyway), then the inverse multiquadric is a good choice.

The *thin-plate spline* radial basis function is

$$\phi(r) = r^2 \log(r/r_0) \tag{3.7.7}$$

with the limiting value $\phi(0) = 0$ assumed. This function has some physical justification in the energy minimization problem associated with warping a thin elastic plate. There is no indication that it is generally better than either of the above forms, however.

The *Gaussian* radial basis function is just what you'd expect,

$$\phi(r) = \exp\left(-\tfrac{1}{2} r^2/r_0^2\right) \tag{3.7.8}$$

The interpolation accuracy using Gaussian basis functions can be very sensitive to r_0, and they are often avoided for this reason. However, for smooth functions and with an optimal r_0, very high accuracy can be achieved. The Gaussian also will extrapolate any function to zero far from the data, and it gets to zero quickly.

Other functions are also in use, for example those of Wendland [6]. There is a large literature in which the above choices for basis functions are tested against specific functional forms or experimental data sets [1,2,7]. Few, if any, general recommendations emerge. We suggest that you try the alternatives in the order listed above, starting with multiquadrics, and that you not omit experimenting with different choices of the scale parameters r_0.

The functions discussed are implemented in code as:

interp_rbf.h

```
struct RBF_multiquadric : RBF_fn {
```
Instantiate this and send to RBF_interp to get multiquadric interpolation.
```
    Doub r02;
    RBF_multiquadric(Doub scale=1.) : r02(SQR(scale)) {}
```
    Constructor argument is the scale factor. See text.
```
    Doub rbf(Doub r) { return sqrt(SQR(r)+r02); }
};

struct RBF_thinplate : RBF_fn {
```
Same as above, but for thin-plate spline.
```
    Doub r0;
    RBF_thinplate(Doub scale=1.) : r0(scale) {}
    Doub rbf(Doub r) { return r <= 0. ? 0. : SQR(r)*log(r/r0); }
};

struct RBF_gauss : RBF_fn {
```
Same as above, but for Gaussian.
```
    Doub r0;
    RBF_gauss(Doub scale=1.) : r0(scale) {}
    Doub rbf(Doub r) { return exp(-0.5*SQR(r/r0)); }
};

struct RBF_inversemultiquadric : RBF_fn {
```
Same as above, but for inverse multiquadric.
```
    Doub r02;
    RBF_inversemultiquadric(Doub scale=1.) : r02(SQR(scale)) {}
    Doub rbf(Doub r) { return 1./sqrt(SQR(r)+r02); }
};
```

Typical use of the objects in this section should look something like this:

```
Int npts=...,ndim=...;
Doub r0=...;
MatDoub pts(npts,ndim);
VecDoub y(npts);
...
RBF_multiquadric multiquadric(r0);
RBF_interp myfunc(pts,y,multiquadric,0);
```

followed by any number of interpolation calls,

```
VecDoub pt(ndim);
Doub val;
...
val = myfunc.interp(pt);
```

3.7.3 Shepard Interpolation

An interesting special case of normalized radial basis function interpolation (equations 3.7.3 and 3.7.4) occurs if the function $\phi(r)$ goes to infinity as $r \to 0$, and is finite (e.g., decreasing) for $r > 0$. In that case it is easy to see that the weights w_i are just equal to the respective function values y_i, and the interpolation formula is simply

$$y(\mathbf{x}) = \frac{\sum_{i=0}^{N-1} y_i \phi(|\mathbf{x} - \mathbf{x}_i|)}{\sum_{i=0}^{N-1} \phi(|\mathbf{x} - \mathbf{x}_i|)} \tag{3.7.9}$$

(with appropriate provision for the limiting case where $\mathbf{x}$ is equal to one of the $\mathbf{x}_i$'s). Note that no solution of linear equations is required. The one-time work is negligible, while the work for each interpolation is $O(N)$, tolerable even for very large N.

Shepard proposed the simple power-law function

$$\phi(r) = r^{-p} \tag{3.7.10}$$

with (typically) $1 < p \lesssim 3$, as well as some more complicated functions with different exponents in an inner and outer region (see [8]). You can see that what is going on is basically interpolation by a nearness-weighted average, with nearby points contributing more strongly than distant ones.

Shepard interpolation is rarely as accurate as the well-tuned application of one of the other radial basis functions, above. On the other hand, it is simple, fast, and often just the thing for quick and dirty applications. It, and variants, are thus widely used.

An implementing object is

interp_rbf.h

```
struct Shep_interp {
```

Object for Shepard interpolation using `n` points in `dim` dimensions. Call constructor once, then `interp` as many times as desired.

```
    Int dim, n;
    const MatDoub &pts;
    const VecDoub &vals;
    Doub pneg;

    Shep_interp(MatDoub_I &ptss, VecDoub_I &valss, Doub p=2.)
    : dim(ptss.ncols()), n(ptss.nrows()) , pts(ptss),
    vals(valss), pneg(-p) {}
```

Constructor. The `n` × `dim` matrix `ptss` inputs the data points, the vector `valss` the function values. Set `p` to the desired exponent. The default value is typical.

```
    Doub interp(VecDoub_I &pt) {
```

Return the interpolated function value at a `dim`-dimensional point `pt`.

```
        Doub r, w, sum=0., sumw=0.;
        if (pt.size() != dim) throw("RBF_interp bad pt size");
        for (Int i=0;i<n;i++) {
            if ((r=rad(&pt[0],&pts[i][0])) == 0.) return vals[i];
            sum += (w = pow(r,pneg));
            sumw += w*vals[i];
        }
        return sumw/sum;
    }

    Doub rad(const Doub *p1, const Doub *p2) {
```

Euclidean distance.

```
        Doub sum = 0.;
        for (Int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
        return sqrt(sum);
    }
};
```

3.7.4 Interpolation by Kriging

Kriging is a technique named for South African mining engineer D.G. Krige. It is basically a form of linear prediction (§13.6), also known in different communities as *Gauss-Markov estimation* or *Gaussian process regression*.

Kriging can be either an interpolation method or a fitting method. The distinction between the two is whether the fitted/interpolated function goes exactly through all the input data points (interpolation), or whether it allows measurement errors to be specified and then "smooths" to get a statistically better predictor that does not

generally go through the data points (does not "honor the data"). In this section we consider only the former case, that is, interpolation. We will return to the latter case in §15.9.

At this point in the book, it is beyond our scope to derive the equations for kriging. You can turn to §13.6 to get a flavor, and look to references [9,10,11] for details. To use kriging, you must be able to estimate the mean square variation of your function $y(\mathbf{x})$ as a function of offset distance $\mathbf{r}$, a so-called *variogram*,

$$v(\mathbf{r}) \sim \tfrac{1}{2}\left\langle [y(\mathbf{x}+\mathbf{r}) - y(\mathbf{x})]^2 \right\rangle \tag{3.7.11}$$

where the average is over all $\mathbf{x}$ with fixed $\mathbf{r}$. If this seems daunting, don't worry. For interpolation, even very crude variogram estimates work fine, and we will give below a routine to estimate $v(\mathbf{r})$ from your input data points $\mathbf{x}_i$ and $y_i = y(\mathbf{x}_i)$, $i = 0, \ldots, N-1$, automatically. One usually takes $v(\mathbf{r})$ to be a function only of the magnitude $r = |\mathbf{r}|$ and writes it as $v(r)$.

Let v_{ij} denote $v(|\mathbf{x}_i - \mathbf{x}_j|)$, where i and j are input points, and let v_{*j} denote $v(|\mathbf{x}_* - \mathbf{x}_j|)$, $\mathbf{x}_*$ being a point at which we want an interpolated value $y(\mathbf{x}_*)$. Now define two vectors of length $N+1$,

$$\begin{aligned} \mathbf{Y} &= (y_0, y_1, \ldots, y_{N-1}, 0) \\ \mathbf{V}_* &= (v_{*1}, v_{*2}, \ldots, v_{*,N-1}, 1) \end{aligned} \tag{3.7.12}$$

and an $(N+1) \times (N+1)$ symmetric matrix,

$$\mathbf{V} = \begin{pmatrix} v_{00} & v_{01} & \ldots & v_{0,N-1} & 1 \\ v_{10} & v_{11} & \ldots & v_{1,N-1} & 1 \\ & & \ldots & & \ldots \\ v_{N-1,0} & v_{N-1,1} & \ldots & v_{N-1,N-1} & 1 \\ 1 & 1 & \ldots & 1 & 0 \end{pmatrix} \tag{3.7.13}$$

Then the kriging interpolation estimate $\hat{y}_* \approx y(\mathbf{x}_*)$ is given by

$$\hat{y}_* = \mathbf{V}_* \cdot \mathbf{V}^{-1} \cdot \mathbf{Y} \tag{3.7.14}$$

and its variance is given by

$$\mathrm{Var}(\hat{y}_*) = \mathbf{V}_* \cdot \mathbf{V}^{-1} \cdot \mathbf{V}_* \tag{3.7.15}$$

Notice that if we compute, once, the LU decomposition of $\mathbf{V}$, and then backsubstitute, once, to get the vector $\mathbf{V}^{-1} \cdot \mathbf{Y}$, then the individual interpolations cost only $O(N)$: Compute the vector $\mathbf{V}_*$ and take a vector dot product. On the other hand, every computation of a variance, equation (3.7.15), requires an $O(N^2)$ backsubstitution.

As an aside (if you have looked ahead to §13.6) the purpose of the extra row and column in $\mathbf{V}$, and extra last components in $\mathbf{V}_*$ and $\mathbf{Y}$, is to automatically calculate, and correct for, an appropriately weighted average of the data, and thus to make equation (3.7.14) an unbiased estimator.

Here is an implementation of equations (3.7.12) – (3.7.15). The constructor does the one-time work, while the two overloaded `interp` methods calculate either an interpolated value or else a value and a standard deviation (square root of the variance). You should leave the optional argument `err` set to the default value of `NULL` until you read §15.9.

krig.h
```
template<class T>
struct Krig {
```
Object for interpolation by kriging, using npt points in ndim dimensions. Call constructor once, then interp as many times as desired.
```
    const MatDoub &x;
    const T &vgram;
    Int ndim, npt;
    Doub lastval, lasterr;
    VecDoub y,dstar,vstar,yvi;
    MatDoub v;
    LUdcmp *vi;
```
Most recently computed value and (if computed) error.
```
    Krig(MatDoub_I &xx, VecDoub_I &yy, T &vargram, const Doub *err=NULL)
    : x(xx),vgram(vargram),npt(xx.nrows()),ndim(xx.ncols()),dstar(npt+1),
    vstar(npt+1),v(npt+1,npt+1),y(npt+1),yvi(npt+1) {
```
Constructor. The npt × ndim matrix xx inputs the data points, the vector yy the function values. vargram is the variogram function or functor. The argument err is not used for interpolation; see §15.9.
```
        Int i,j;
        for (i=0;i<npt;i++) {
            y[i] = yy[i];
            for (j=i;j<npt;j++) {
                v[i][j] = v[j][i] = vgram(rdist(&x[i][0],&x[j][0]));
            }
            v[i][npt] = v[npt][i] = 1.;
        }
        v[npt][npt] = y[npt] = 0.;
        if (err) for (i=0;i<npt;i++) v[i][i] -= SQR(err[i]);
        vi = new LUdcmp(v);
        vi->solve(y,yvi);
    }
    ~Krig() { delete vi; }
```
Fill $\mathbf{Y}$ and $\mathbf{V}$.

§15.9.
```
    Doub interp(VecDoub_I &xstar) {
```
Return an interpolated value at the point xstar.
```
        Int i;
        for (i=0;i<npt;i++) vstar[i] = vgram(rdist(&xstar[0],&x[i][0]));
        vstar[npt] = 1.;
        lastval = 0.;
        for (i=0;i<=npt;i++) lastval += yvi[i]*vstar[i];
        return lastval;
    }

    Doub interp(VecDoub_I &xstar, Doub &esterr) {
```
Return an interpolated value at the point xstar, and return its estimated error as esterr.
```
        lastval = interp(xstar);
        vi->solve(vstar,dstar);
        lasterr = 0;
        for (Int i=0;i<=npt;i++) lasterr += dstar[i]*vstar[i];
        esterr = lasterr = sqrt(MAX(0.,lasterr));
        return lastval;
    }

    Doub rdist(const Doub *x1, const Doub *x2) {
```
Utility used internally. Cartesian distance between two points.
```
        Doub d=0.;
        for (Int i=0;i<ndim;i++) d += SQR(x1[i]-x2[i]);
        return sqrt(d);
    }
};
```

The constructor argument vgram, the variogram function, can be either a func-

tion or functor (§1.3.3). For interpolation, you can use a `Powvargram` object that fits a simple model

$$v(r) = \alpha r^{\beta} \tag{3.7.16}$$

where β is considered fixed and α is fitted by unweighted least squares over all pairs of data points i and j. We'll get more sophisticated about variograms in §15.9; but for interpolation, excellent results can be obtained with this simple choice. The value of β should be in the range $1 \leq \beta < 2$. A good general choice is 1.5, but for functions with a strong linear trend, you may want to experiment with values as large as 1.99. (The value 2 gives a degenerate matrix and meaningless results.) The optional argument nug will be explained in §15.9.

krig.h

```
struct Powvargram {
```
Functor for variogram $v(r) = \alpha r^{\beta}$, where β is specified, α is fitted from the data.
```
    Doub alph, bet, nugsq;

    Powvargram(MatDoub_I &x, VecDoub_I &y, const Doub beta=1.5, const Doub nug=0.)
    : bet(beta), nugsq(nug*nug) {
```
    Constructor. The `npt` × `ndim` matrix `x` inputs the data points, the vector `y` the function values, `beta` the value of β. For interpolation, the default value of `beta` is usually adequate. For the (rare) use of `nug` see §15.9.
```
        Int i,j,k,npt=x.nrows(),ndim=x.ncols();
        Doub rb,num=0.,denom=0.;
        for (i=0;i<npt;i++) for (j=i+1;j<npt;j++) {
            rb = 0.;
            for (k=0;k<ndim;k++) rb += SQR(x[i][k]-x[j][k]);
            rb = pow(rb,0.5*beta);
            num += rb*(0.5*SQR(y[i]-y[j]) - nugsq);
            denom += SQR(rb);
        }
        alph = num/denom;
    }

    Doub operator() (const Doub r) const {return nugsq+alph*pow(r,bet);}
};
```

Sample code for interpolating on a set of data points is

```
MatDoub x(npts,ndim);
VecDoub y(npts), xstar(ndim);
...
Powvargram vgram(x,y);
Krig<Powvargram> krig(x,y,vgram);
```

followed by any number of interpolations of the form

```
ystar = krig.interp(xstar);
```

Be aware that while the interpolated values are quite insensitive to the variogram model, the estimated errors are rather sensitive to it. You should thus consider the error estimates as being order of magnitude only. Since they are also relatively expensive to compute, their value in this application is not great. They will be much more useful in §15.9, when our model includes measurement errors.

3.7.5 Curve Interpolation in Multidimensions

A different kind of interpolation, worth a brief mention here, is when you have an ordered set of N tabulated points in n dimensions that lie on a one-dimensional curve, $\mathbf{x}_0, \ldots \mathbf{x}_{N-1}$, and you want to interpolate other values along the curve. Two

cases worth distinguishing are: (i) The curve is an open curve, so that $\mathbf{x}_0$ and $\mathbf{x}_{N-1}$ represent endpoints. (ii) The curve is a closed curve, so that there is an implied curve segment connecting $\mathbf{x}_{N-1}$ back to $\mathbf{x}_0$.

A straightforward solution, using methods already at hand, is first to approximate distance along the curve by the sum of chord lengths between the tabulated points, and then to construct spline interpolations for each of the coordinates, $0, \ldots, n-1$, as a function of that parameter. Since the derivative of any single coordinate with respect to arc length can be no greater than 1, it is guaranteed that the spline interpolations will be well-behaved.

Probably 90% of applications require nothing more complicated than the above. If you are in the unhappy 10%, then you will need to learn about *Bézier curves*, *B-splines*, and *interpolating splines* more generally [12,13,14]. For the happy majority, an implementation is

interp_curve.h

```
struct Curve_interp {
```
Object for interpolating a curve specified by n points in dim dimensions.
```
    Int dim, n, in;
    Bool cls;                                   Set if a closed curve.
    MatDoub pts;
    VecDoub s;
    VecDoub ans;
    NRvector<Spline_interp*> srp;

    Curve_interp(MatDoub &ptsin, Bool close=0)
    : n(ptsin.nrows()), dim(ptsin.ncols()), in(close ? 2*n : n),
    cls(close), pts(dim,in), s(in), ans(dim), srp(dim) {
```
Constructor. The n × dim matrix ptsin inputs the data points. Input close as 0 for an open curve, 1 for a closed curve. (For a closed curve, the last data point should *not* duplicate the first — the algorithm will connect them.)
```
        Int i,ii,im,j,ofs;
        Doub ss,soff,db,de;
        ofs = close ? n/2 : 0;                  The trick for closed curves is to duplicate half a
        s[0] = 0.;                                  period at the beginning and end, and then
        for (i=0;i<in;i++) {                        use the middle half of the resulting spline.
            ii = (i-ofs+n) % n;
            im = (ii-1+n) % n;
            for (j=0;j<dim;j++) pts[j][i] = ptsin[ii][j];       Store transpose.
            if (i>0) {                          Accumulate arc length.
                s[i] = s[i-1] + rad(&ptsin[ii][0],&ptsin[im][0]);
                if (s[i] == s[i-1]) throw("error in Curve_interp");
```
Consecutive points may not be identical. For a closed curve, the last data point should *not* duplicate the first.
```
            }
        }
        ss = close ? s[ofs+n]-s[ofs] : s[n-1]-s[0];  Rescale parameter so that the
        soff = s[ofs];                              interval [0,1] is the whole curve (or one period).
        for (i=0;i<in;i++) s[i] = (s[i]-soff)/ss;
        for (j=0;j<dim;j++) {                   Construct the splines using endpoint derivatives.
            db = in < 4 ? 1.e99 : fprime(&s[0],&pts[j][0],1);
            de = in < 4 ? 1.e99 : fprime(&s[in-1],&pts[j][in-1],-1);
            srp[j] = new Spline_interp(s,&pts[j][0],db,de);
        }
    }
    ~Curve_interp() {for (Int j=0;j<dim;j++) delete srp[j];}

    VecDoub &interp(Doub t) {
```
Interpolate a point on the stored curve. The point is parameterized by t, in the range [0,1]. For open curves, values of t outside this range will return extrapolations (dangerous!). For closed curves, t is periodic with period 1.

```
        if (cls) t = t - floor(t);
        for (Int j=0;j<dim;j++) ans[j] = (*srp[j]).interp(t);
        return ans;
    }

    Doub fprime(Doub *x, Doub *y, Int pm) {
    Utility for estimating the derivatives at the endpoints. x and y point to the abscissa and
    ordinate of the endpoint. If pm is +1, points to the right will be used (left endpoint); if it
    is −1, points to the left will be used (right endpoint). See text, below.
        Doub s1 = x[0]-x[pm*1], s2 = x[0]-x[pm*2], s3 = x[0]-x[pm*3],
            s12 = s1-s2, s13 = s1-s3, s23 = s2-s3;
        return -(s1*s2/(s13*s23*s3))*y[pm*3]+(s1*s3/(s12*s2*s23))*y[pm*2]
            -(s2*s3/(s1*s12*s13))*y[pm*1]+(1./s1+1./s2+1./s3)*y[0];
    }

    Doub rad(const Doub *p1, const Doub *p2) {
    Euclidean distance.
        Doub sum = 0.;
        for (Int i=0;i<dim;i++) sum += SQR(p1[i]-p2[i]);
        return sqrt(sum);
    }

};
```

The utility routine `fprime` estimates the derivative of a function at a tabulated abscissa x_0 using four consecutive tabulated abscissa-ordinate pairs, $(x_0, y_0), \ldots, (x_3, y_3)$. The formula for this, readily derived by power-series expansion, is

$$y_0' = -C_0 y_0 + C_1 y_1 - C_2 y_2 + C_3 y_3 \tag{3.7.17}$$

where

$$\begin{aligned} C_0 &= \frac{1}{s_1} + \frac{1}{s_2} + \frac{1}{s_3} \\ C_1 &= \frac{s_2 s_3}{s_1(s_2 - s_1)(s_3 - s_1)} \\ C_2 &= \frac{s_1 s_3}{(s_2 - s_1)s_2(s_3 - s_2)} \\ C_3 &= \frac{s_1 s_2}{(s_3 - s_1)(s_3 - s_2)s_3} \end{aligned} \tag{3.7.18}$$

with

$$\begin{aligned} s_1 &\equiv x_1 - x_0 \\ s_2 &\equiv x_2 - x_0 \\ s_3 &\equiv x_3 - x_0 \end{aligned} \tag{3.7.19}$$

CITED REFERENCES AND FURTHER READING:

Buhmann, M.D. 2003, *Radial Basis Functions: Theory and Implementations* (Cambridge, UK: Cambridge University Press).[1]

Powell, M.J.D. 1992, "The Theory of Radial Basis Function Approximation" in *Advances in Numerical Analysis II: Wavelets, Subdivision Algorithms and Radial Functions*, ed. W. A. Light (Oxford: Oxford University Press), pp. 105–210.[2]

Wikipedia. 2007+, "Radial Basis Functions," at `http://en.wikipedia.org/`.[3]

Beatson, R.K. and Greengard, L. 1997, "A Short Course on Fast Multipole Methods", in *Wavelets, Multilevel Methods and Elliptic PDEs*, eds. M. Ainsworth, J. Levesley, W. Light, and M. Marletta (Oxford: Oxford University Press), pp. 1–37.[4]